

INTERNSHIP PROJECT REPORT

An AI-Powered Study Assistant Web Application

STUDY WITH BUDDY

Submitted as part of the AI Engineer Internship at
SHADOWFOX

Submitted by

ILAMARAN R

B.Tech, Information Technology — 3rd Year
PKIET

Internship Duration

August 1, 2026 – September 2, 2026

Table of Contents

Abstract.....	3
1. Introduction.....	4
2. Problem Statement.....	5
3. Objectives.....	5
4. Technologies Used.....	6
5. System Requirements.....	6
6. System Architecture / Workflow.....	8
7. Project Features and Modules.....	9
8. Implementation / Methodology.....	10
9. Screenshots and UI Explanation.....	12
10. Testing and Results.....	14
11. Advantages.....	15
12. Limitations.....	15
13. Future Enhancements.....	16
14. Conclusion.....	16
15. References.....	17

Abstract

This report documents “Study With Buddy,” a beginner-level AI-powered study assistant web application developed during a one-month AI Engineer internship at SHADOWFOX (August 1, 2026 – September 2, 2026). The application allows a student to paste study notes or a draft answer into a web page and choose one of three actions — Summarize, Short Quiz, or Better Answer — which are then processed by Google’s Gemini API to generate a useful, formatted response.

The project was built using Python and the Flask web framework for the backend, plain HTML, CSS and JavaScript for the frontend, and the google-genai SDK to communicate with the Gemini language model (gemini-3.6-flash). The purpose of this project was to gain hands-on, practical experience in integrating a generative AI API into a simple full-stack web application, covering backend routing, prompt design, response handling, and basic frontend interactivity. The resulting application is a small but complete demonstration of how a generative AI service can be wrapped inside an accessible, easy-to-use tool for everyday student use.

1. Introduction

Artificial Intelligence has become increasingly accessible to developers through hosted large language model (LLM) APIs, which allow even beginner programmers to build applications that generate human-quality text. As part of a beginner-level AI Engineer internship at SHADOWFOX, this project was undertaken to explore how such an API can be integrated into a simple, real-world-style web application aimed at helping students study more effectively. “Study With Buddy” is a lightweight Flask web application that takes study notes or a draft answer typed in by the user and sends them, along with a task-specific instruction, to Google’s Gemini API. The API’s response is then formatted and displayed back to the user on a separate results page. The project intentionally keeps its scope small and focused, which is appropriate for a one-month, beginner-level internship engagement, while still covering the essential building blocks of a modern AI-integrated web application: backend routing, external API integration, prompt engineering, and basic UI/UX design.

1.1 Internship Overview

Company	SHADOWFOX
Internship Domain	AI Engineer
Duration	August 1, 2026 – September 2, 2026 (approx. 1 month)
Internship Level	Beginner
Intern Name	Ilamaran R
College	PKIET
Degree / Department	B.Tech, Information Technology (3rd Year)

2. Problem Statement

Students frequently deal with large volumes of study material — lecture notes, textbook excerpts, and self-written answers — but do not always have a quick way to condense this material, test their own understanding of it, or improve the quality of their written answers. Manually summarizing long notes is time-consuming, creating a self-test quiz from one's own notes requires extra effort that is often skipped, and getting objective feedback on a draft answer typically requires a teacher, tutor, or peer who may not always be available.

This project addresses that gap by providing a simple, single-page web tool where a student can paste their notes or draft answer and instantly receive one of three AI-generated outputs — a summary, a short self-test quiz, or an improved version of their answer — without needing any external tools, accounts, or manual effort beyond pasting text and clicking a button.

3. Objectives

- To design and develop a simple, functional Flask-based web application that integrates a generative AI API (Gemini) for real-world use.
- To provide students with three practical AI-assisted study actions: summarizing notes, generating a self-test quiz, and improving a draft answer.
- To gain hands-on experience with prompt design — writing clear, action-specific instructions that guide the AI model toward the desired output format.
- To build a clean, minimal, and responsive user interface using HTML, CSS, and JavaScript that is easy for a student to use without any training.
- To handle the AI response safely, converting Markdown-formatted AI output into readable HTML, and to gracefully handle API errors instead of crashing the application.

- To understand and practice the basic architecture of a full-stack web application: routing, server-side rendering with Jinja2 templates, and client-side form handling.

4. Technologies Used

The project was intentionally built with a small, beginner-friendly technology stack, consistent with the scope of a one-month beginner-level internship task.

Category	Technology / Tool
Backend Language	Python 3
Web Framework	Flask
Generative AI API	Google Gemini API, model: gemini-3.6-flash
AI SDK	google-genai (Python client library)
Frontend	HTML5, CSS3, JavaScript (vanilla JS)
Templating Engine	Jinja2 (Flask's built-in templating engine)
Markdown Rendering	Python markdown library (with the 'extra' extension)
Environment Configuration	python-dotenv (for loading the Gemini API key from a .env file)
Development Server	Flask built-in development server (app.run(debug=True))

5. System Requirements

5.1 Hardware Requirements

- Processor: Intel Core i3 or higher (or equivalent)
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 100 MB free space for project files and dependencies
- Internet Connection: Required, for communicating with the Gemini API

5.2 Software Requirements

- Operating System: Windows / macOS / Linux (cross-platform)
- Python 3.9 or later
- pip (Python package manager)
- A modern web browser (Chrome, Edge, Firefox, etc.)
- Code editor (e.g., VS Code) — used during development
- A valid Google Gemini API key

5.3 Python Dependencies

As listed in requirements.txt:

- flask
- google-genai
- markdown
- python-dotenv

6. System Architecture / Workflow

Study With Buddy follows a simple client-server architecture typical of a small Flask web application. The browser (client) sends the user's notes and selected action to the Flask backend, which builds an action-specific prompt, sends it to the Gemini API, converts the returned text into HTML, and renders the result back to the user.

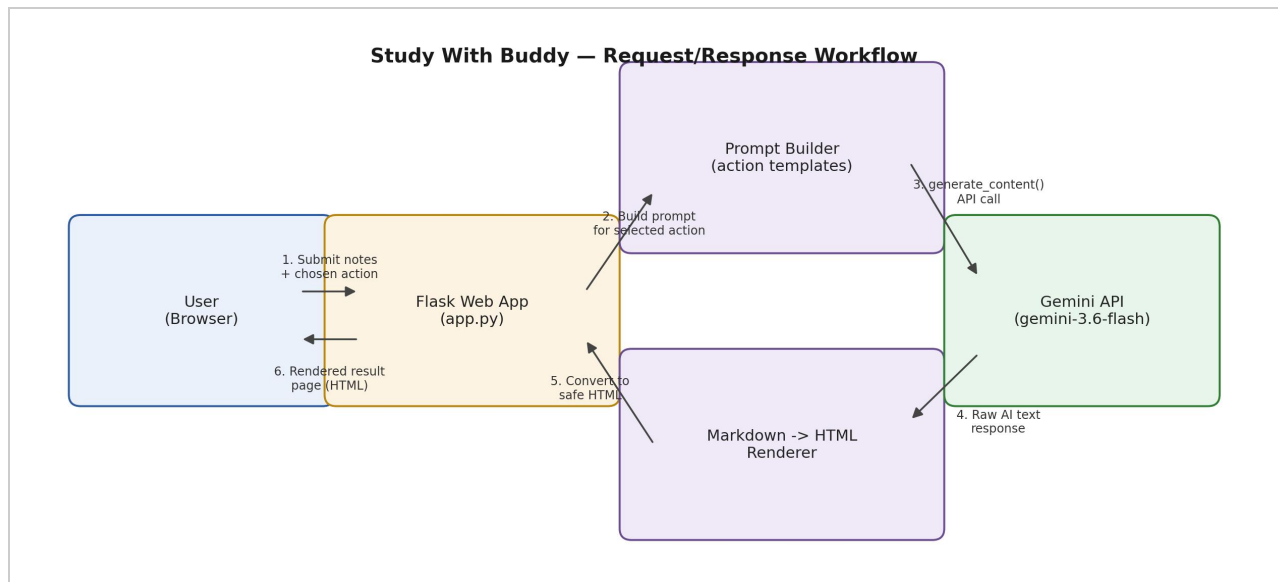


Figure 1: Request/response workflow of the Study With Buddy application

6.1 Workflow Steps

- The user opens the home page (index.html) and pastes their study notes or draft answer into a text area.
- The user selects one of three actions: Summarize, Short Quiz, or Better Answer, and submits the form.
- The Flask route /result (POST) receives the notes and the chosen action.
- The backend looks up the matching prompt template for that action from the ACTIONS dictionary and formats it with the user's notes.
- The formatted prompt is sent to the Gemini API using `client.models.generate_content()` with the model `gemini-3.6-flash`.
- The raw text response from Gemini is escaped and converted from Markdown into HTML using the markdown library.
- The formatted HTML result is rendered inside `result.html` and displayed to the user, along with options to copy the result or try another action.
- If the API call fails for any reason, a friendly fallback error message is shown instead of the application crashing.

7. Project Features and Modules

7.1 Summarize

Condenses long study notes into a short, well-organized summary using clear paragraphs and bullet points, based strictly on the content of the notes provided (the prompt explicitly instructs the model not to add outside information).

7.2 Short Quiz

Automatically generates a 5-question self-test quiz based on the pasted notes, with a separate, clearly numbered 'Answers' section at the end so the student can test themselves before checking the answers.

7.3 Better Answer

Takes a student's draft answer to a study question and rewrites it to be clearer, more complete, and better organized, while preserving the original meaning. The AI also briefly explains what was changed and why, helping the student learn from the revision rather than just receiving a replacement.

7.4 Supporting Modules

- Prompt template system (ACTIONS dictionary) — keeps each action's instruction, label, and result title organized in one place.
- Markdown-to-HTML converter (text_to_html function) — escapes the AI's raw text for safety and renders it as clean, readable HTML.
- Result page actions — 'Copy to clipboard' and 'Try another action' buttons for a smoother study workflow.
- Basic error handling — displays a friendly message if the Gemini API call fails instead of showing a server error.

8. Implementation / Methodology

The application was implemented as a small full-stack Flask project with a clear separation between the backend logic (app.py), the presentation layer (Jinja2 templates), and static assets (CSS and JavaScript).

8.1 Backend (app.py)

- A Flask app is initialized and a Gemini client is created using an API key loaded from a .env file via python-dotenv.
- An ACTIONS dictionary defines the three supported actions (summarize, quiz, better_answer), each with a display label, a result page title, and a prompt template containing a {notes} placeholder.
- The index route ('/') simply renders the notes-input page (index.html).
- The result route ('/result', POST) reads the submitted notes and action from the form, validates that both are present and that the action is recognized, builds the final prompt by formatting the matching template, and calls the Gemini API.
- The AI's response text is passed through the text_to_html() helper function, which HTML-escapes the content first (for safety) and then converts it from Markdown into HTML using the markdown library's 'extra' extension, before being rendered on the result page.

8.2 Frontend (templates and static files)

- index.html — contains the notes textarea, a live word counter, and three submit buttons (Summarize, Short Quiz, Better Answer).
- result.html — displays the AI-generated, HTML-rendered result along with the action label, a 'Copy to clipboard' button, and a 'Try another action' link back to the home page.
- style.css — provides a consistent dark, desk-lamp-inspired visual theme across both pages.
- script.js — handles client-side interactivity, including the word counter and safely disabling the submit button after a click (using a short setTimeout delay so the button's value is still included in the submitted form data).

8.3 Development Environment

The project was developed and tested locally using Flask's built-in development server, run from a Python virtual environment (.venv), as shown in the terminal output below.

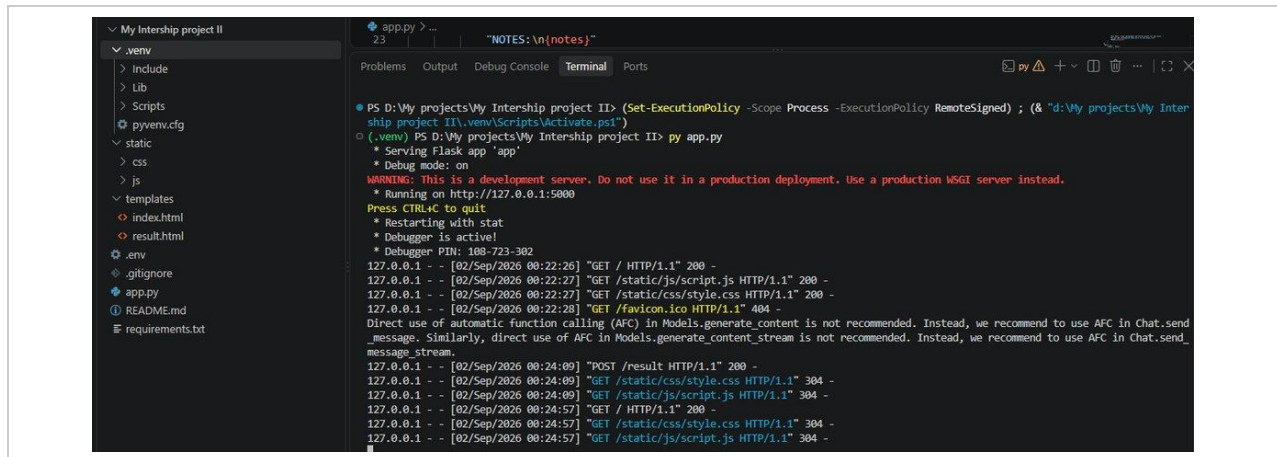


Figure 2: Project structure and Flask development server running locally in VS Code

9. Screenshots and UI Explanation

9.1 Home Page — Notes Input

The home page presents a single, focused text area labelled 'Your Notes' where the student pastes their study notes or draft answer. A live word counter is shown in the top-right of the input box, along with a hint that the tool works best with 50–2,000 words at a time. Below the text area are three clearly labelled action buttons — Summarize, Short Quiz, and Better Answer — letting the student choose the type of AI assistance they need.

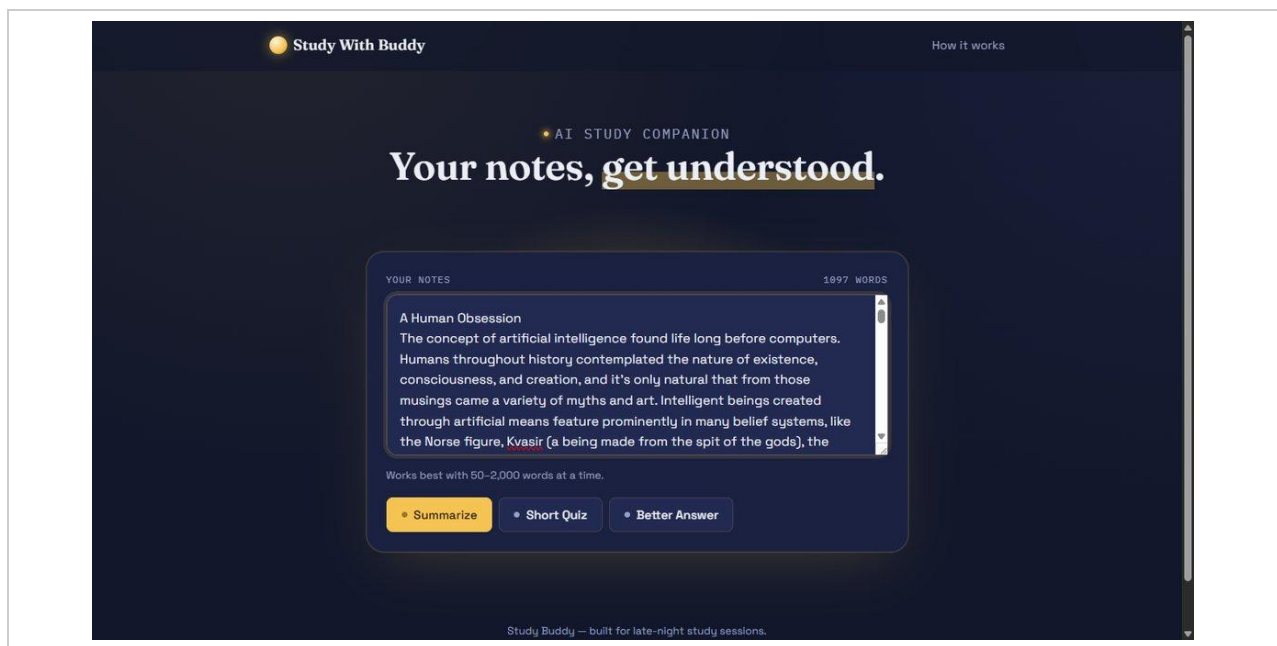


Figure 3: Home page — notes input with three action buttons

9.2 Result Page — AI-Generated Summary

After submitting notes with the ‘Summarize’ action, the result page displays a badge indicating the action performed (‘SUMMARY’), a heading (‘Here’s your summary’), and the AI-generated content rendered as formatted HTML with headings and bullet points for readability. A ‘Back to your notes’ link and a ‘New notes’ shortcut are available at the top of the page.

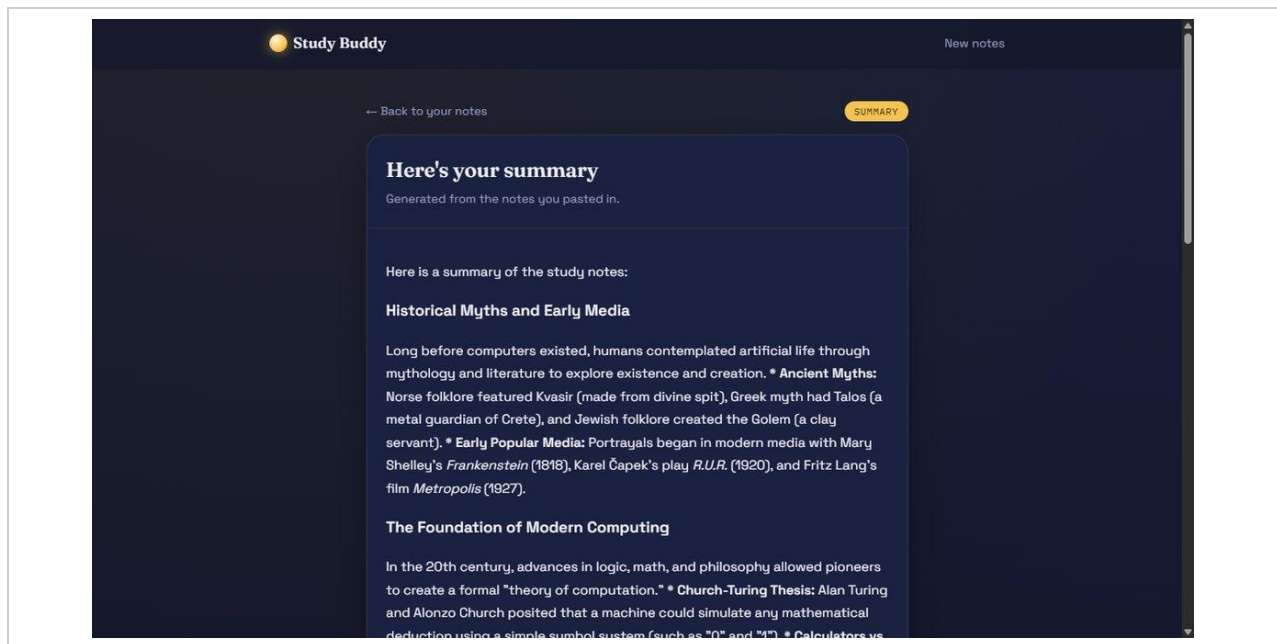


Figure 4: Result page — top section showing the AI-generated summary

9.3 Result Page — Actions

At the bottom of the result page, the student can either copy the generated content to the clipboard for use elsewhere (e.g., in their own notes document) or click ‘Try another action’ to run a different action (Quiz or Better Answer) on the same notes.

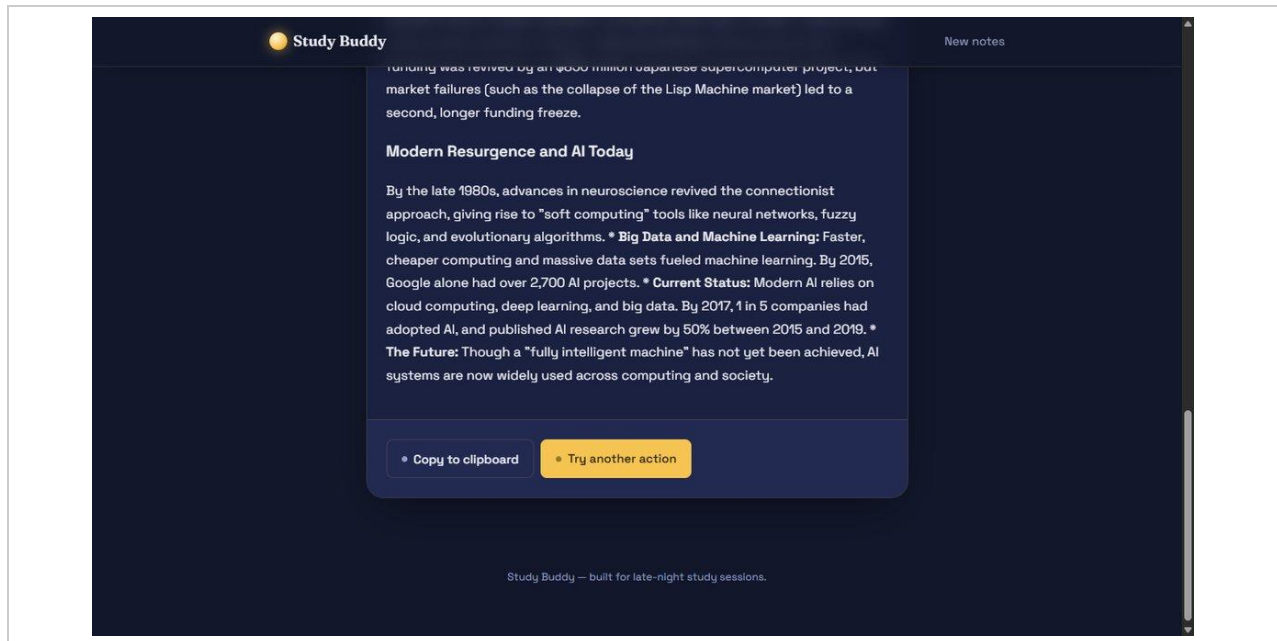


Figure 5: Result page — bottom section with ‘Copy to clipboard’ and ‘Try another action’ options

10. Testing and Results

Testing was carried out manually during development by running the Flask development server locally and interacting with the application through the browser, as is typical and appropriate for a beginner-level project of this scope.

10.1 Test Approach

- Ran the application locally using ‘python app.py’ and accessed it at <http://127.0.0.1:5000>.
- Submitted real study notes (e.g., an article on the history of artificial intelligence) using each of the three actions to confirm correct behavior.
- Verified that the Summarize action returns a well-organized summary with headings and bullet points, as seen in the terminal logs and result page (Figures 4–5).
- Verified that the Short Quiz action returns exactly five numbered questions followed by a separate, correctly numbered Answers section.
- Verified that the Better Answer action rewrites a draft answer and appends a short note on what was changed.
- Tested edge cases such as submitting the form with an empty notes field, which correctly redirects back to the home page instead of calling the API.
- Confirmed via the terminal logs that POST requests to /result return a 200 status code, indicating successful processing (see Figure 2).

10.2 Results Summary

Feature Tested	Observed Result
Summarize	Produces a concise, structured summary using only information present in the notes
Short Quiz	Produces exactly 5 numbered questions with a matching, correctly numbered Answers section
Better Answer	Rewrites the draft answer clearly and lists the changes made
Empty input handling	Redirects to the home page without calling the API
API failure handling	Displays a friendly fallback error message instead of crashing

11. Advantages

- Saves students time by instantly summarizing long notes into an easy-to-review format.
- Encourages active recall and self-assessment through automatically generated quizzes.
- Helps students improve their own writing and answer structure through AI-guided rewriting with explanations.
- Very simple, distraction-free interface that requires no training to use.
- Lightweight technology stack that is easy to set up, run, and maintain, even for a beginner developer.
- Reasonably robust error handling ensures the app does not crash if the AI service is temporarily unavailable.

12. Limitations

- The application does not store or save previous notes, summaries, or quizzes — it is stateless and single-session only.
- There is no user authentication or account system, so results cannot be saved to a personal history.
- Requires an active internet connection and a valid Gemini API key at all times; it cannot function offline.
- Quiz and summary quality is entirely dependent on the AI model's output and the clarity of the notes provided by the user.
- The app is currently intended for a single user at a time via the Flask development server and has not been deployed with a production-grade WSGI server.
- No automated unit or integration tests were written; testing was performed manually.

13. Future Enhancements

- Add user accounts and a history feature so students can revisit past summaries and quizzes.
- Support file uploads (e.g., PDF or DOCX notes) instead of requiring notes to be pasted manually.
- Add an interactive quiz mode where questions are answered directly in the browser with instant scoring.
- Deploy the application using a production-grade WSGI server (e.g., Gunicorn) behind a proper host, instead of the Flask development server.
- Add support for additional study actions, such as flashcard generation or concept explanation.
- Introduce basic automated testing to validate prompt behavior and error handling.

14. Conclusion

The “Study With Buddy” project successfully demonstrates how a beginner developer can integrate a generative AI API into a functional, full-stack web application within a short, one-month internship timeframe. By combining a simple Flask backend, action-specific prompt engineering, and a clean, minimal frontend, the application delivers three genuinely useful study features — summarization, quiz generation, and answer improvement — in a single, easy-to-use tool.

Working on this project provided practical, hands-on experience with core AI engineering concepts appropriate to a beginner level, including designing clear prompts for a language model, handling API responses safely, and building a basic but complete user interface around an AI service. While the application has clear limitations in scope, such as the lack of persistent storage and user accounts, it fulfills its intended purpose as a beginner-level internship project and provides a solid foundation for the future enhancements outlined above.

15. References

- Google AI for Developers — Gemini API Documentation: <https://ai.google.dev/gemini-api/docs>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Python markdown library Documentation: <https://python-markdown.github.io/>
- python-dotenv Documentation: <https://pypi.org/project/python-dotenv/>
- Jinja2 Documentation: <https://jinja.palletsprojects.com/>
- Project source code and README, “Study With Buddy,” developed as part of the SHADOWFOX AI Engineer internship (August–September 2026).